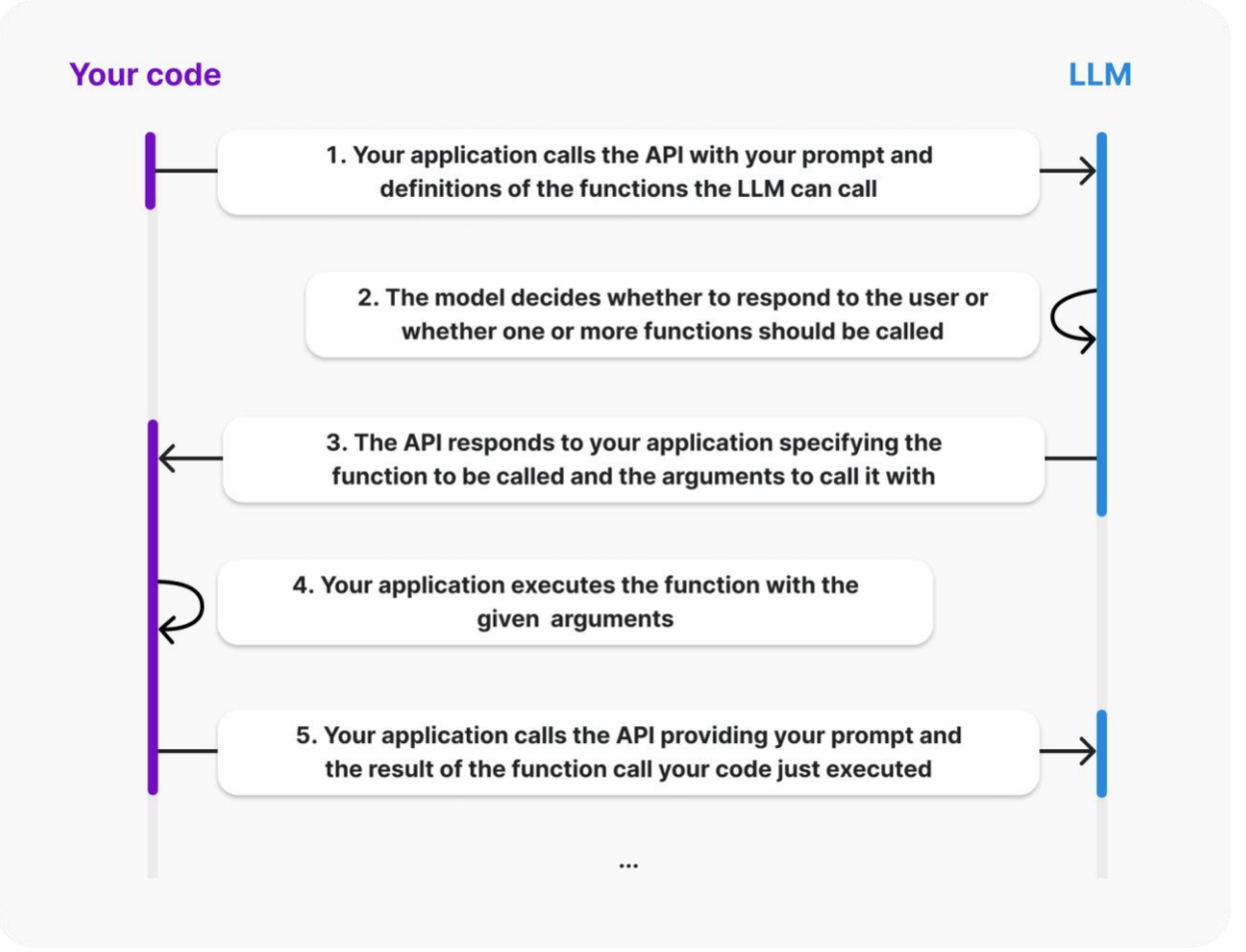
Tool Use

Ronald A. Sumida
rsumida@innovationinsoftware.com

Tool Use (Formerly Function Calling)

- Extends an LLM's powers by making user-defined functions available to it
  - Call external APIs
  - Query a database
  - Perform math
- LLM parses input, tells you which function(s) to call, and provides values for function parameters



Source: https://platform.openai.com/docs/guides/function-calling

Describing a Function to an LLM

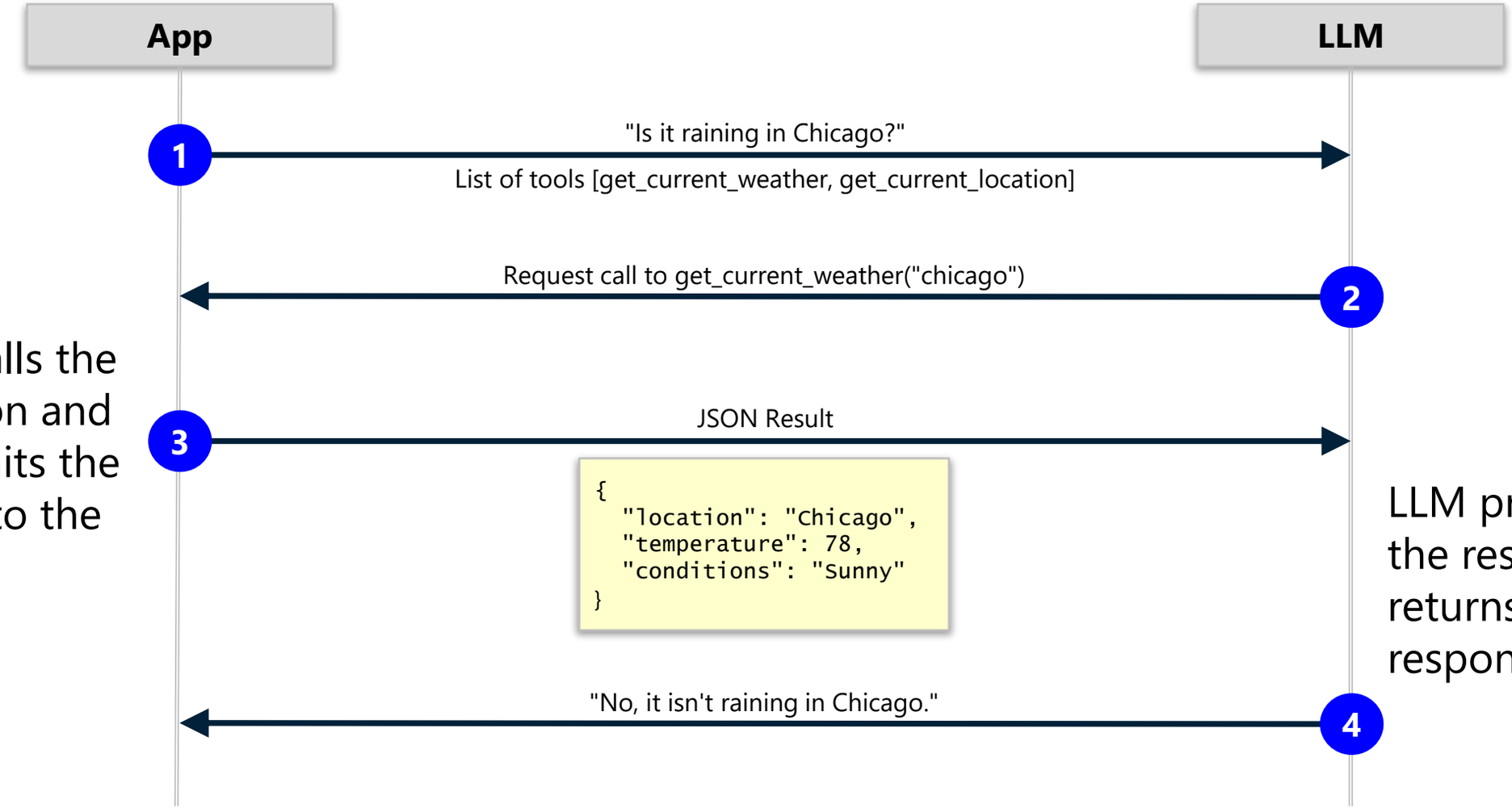
```
weather_tool = {
    'type': 'function',
    'function': {
        'name': 'get_current_weather',
        'description': 'Retrieves the current weather at the specified location',
        'parameters': {
            'type': 'object',
            'properties': {
                'location': {
                    'type': 'string', # number, string, boolean, array, or object
                    'description': 'The location whose weather is to be retrieved.'
                }
            },
            'required': ['location']
        }
    }
}
```

Making a Tool Available to an LLM

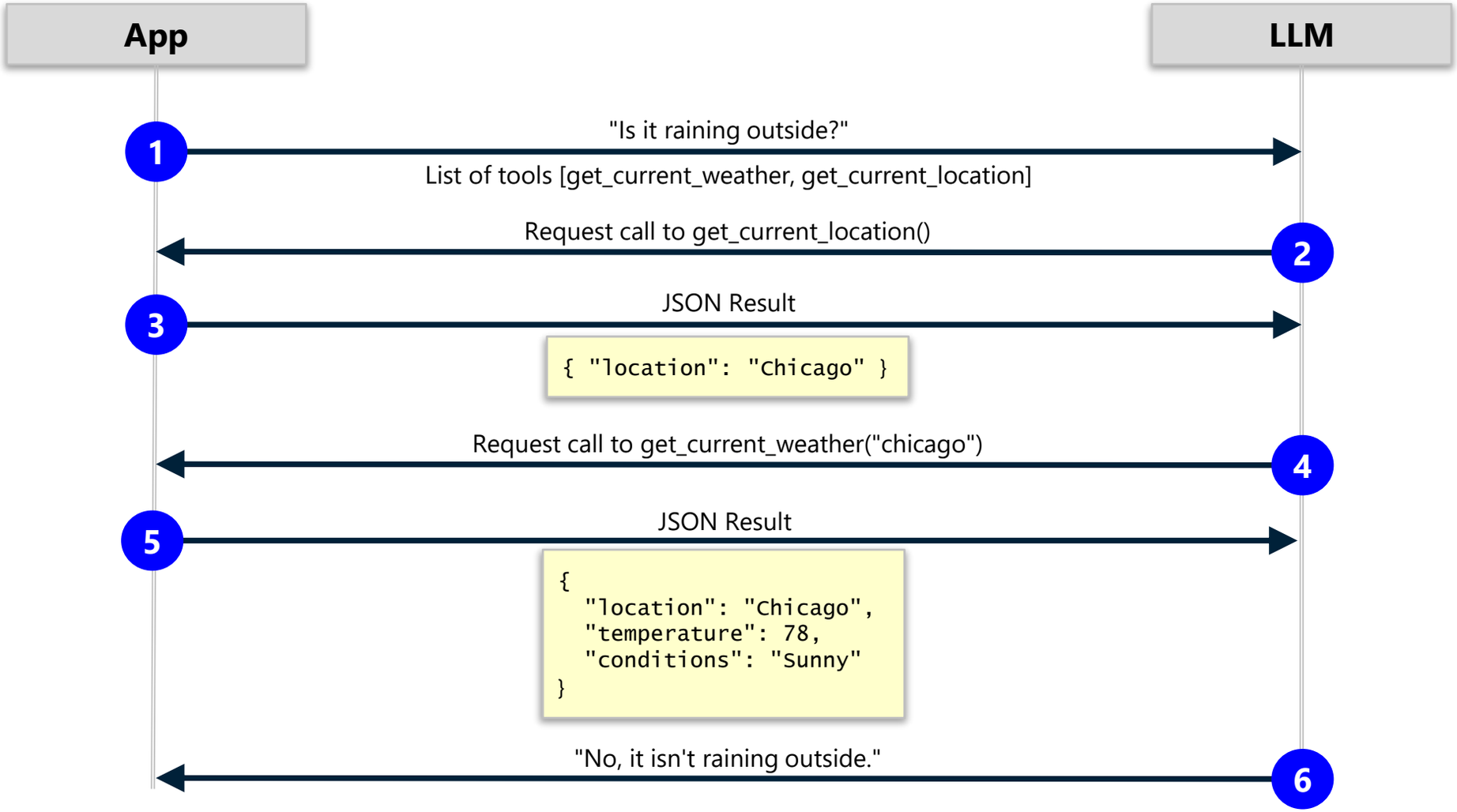
```
client = OpenAI()
messages = [
    { 'role': 'system', 'content': 'You are a helpful assistant.' },
    { 'role': 'user', 'content': 'Is it raining in Chicago?' },
]

response = client.chat.completions.create(
    model='gpt-5.4-mini',
    messages=messages,
    tools=[weather_tool]
)
```

The Tools Protocol



The Tools Protocol, Cont.



Demo

Tool Use

Code Generation and Execution

- LLMs are adept at generating code in more than 40 languages
- Use this capability to allow an LLM to generate charts and graphs, perform mathematical calculations, and more

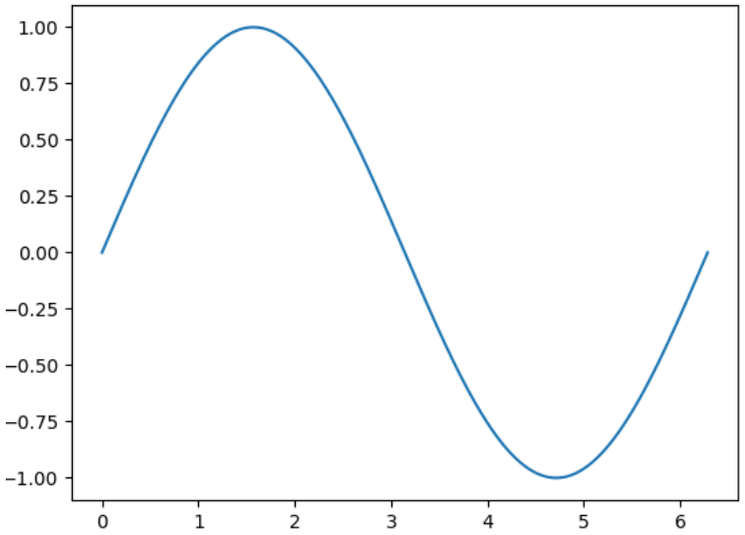
Plot a sine wave.

```
You are an expert
Python programmer.
Generate code to
respond to the
following command:

Plot a sine wave.
```

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2 * np.pi, 500)
plt.plot(x, np.sin(x))
plt.show()
```



Generating Python Code with an LLM

```
prompt = '''
    Generate Python code to respond to the following input: What is 2+2?
    Respond with the code only. Do not use markdown formatting. Do not generate
    code that could be harmful to the computer that it runs on.
    '''


messages = [
    { 'role': 'system', 'content': 'You are an expert Python programmer.' },
    { 'role': 'user', 'content': prompt }
]

client = OpenAI()
response = client.chat.completions.create(model='gpt-5.4-mini', messages=messages)
code = response.choices[0].message.content # print("2+2")
```

Executing LLM-Generated Code in-Process

```
import io, sys

# Redirect STDOUT to capture output
buffer = io.StringIO()
sys.stdout = buffer

# Execute the code
exec(code)

# Restore STDOUT and retrieve the output
sys.stdout = sys.__stdout__
output = buffer.getvalue())
```

Executing LLM-Generated Code in a Subprocess

```
import subprocess

# Execute the code in a subprocess
result = subprocess.run(
    ['python', '-c', code],
    capture_output=True,
    text=True
)

# Retrieve the output
output = result.stdout
```

Sandboxing LLM-Generated Code

- LLM-generated code should never be trusted
  - Running it "on the metal" with **exec** or **subprocess.run** is risky
  - Code could format a hard disk, steal passwords or other secrets, or worse
- "Sandboxing" runs LLM-generated code in an isolated environment
  - Virtual machine
  - Docker container
  - Kubernetes container
- Libraries such as **llm-sandbox** simplify the process

Running Code in a Docker Container

```python
# Make sure Docker Desktop is running before executing this code

from llm_sandbox import SandboxSession

# Run the code in a Docker container instance
with SandboxSession(lang='python') as session:
    result = session.run(code)
    output = result.stdout
```

Using a Prewarmed Container Pool

```python
from llm_sandbox import SandboxSession
from llm_sandbox.pool import create_pool_manager, PoolConfig

# Create a container pool
pool = create_pool_manager(
    backend='docker',
    config=PoolConfig(max_pool_size=3, min_pool_size=1),
    lang='python'
)

# Run the code in a pooled container instance
with SandboxSession(pool=pool, lang='python') as session:
    result = session.run(code)
    output = result.stdout
```

Generating an Image

```python
from llm_sandbox import ArtifactSandboxSession

with ArtifactSandboxSession(lang='python') as session:
    result = session.run(
        code,
        libraries=['numpy', 'matplotlib']
    )

    for plot in result.plots:
        base_64_encoded_image = plot.content_base64
```

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2 * np.pi, 500)
plt.plot(x, np.sin(x))
plt.show()
```

Copying Files To and From a Sandbox

```python
from llm_sandbox import SandboxSession

with SandboxSession(lang='python', keep_template=True) as session:
    # Copy a file from the host to the sandbox
    session.copy_to_runtime('generate_plot.py', '/sandbox/generate_plot.py')

    # Run the copied Python code in the sandbox
    result = session.execute_command('python /sandbox/generate_plot.py "chart.png"')
    print(result)

    # Copy a file from the sandbox to the host
    session.copy_from_runtime('/sandbox/chart.png', 'chart.png')
```

Demo

Code Generation and Execution

Fine-Tuning

- Further trains a pre-trained model on task-specific examples

Fine-tune when...

- Output format or behavior must be highly consistent
- Style, tone, or terminology must persist across requests
- Prompting alone cannot reliably meet evaluated requirements

Use retrieval or tools for...

- Current or frequently changing information
- Proprietary documents and factual knowledge
- Information that needs traceability or citations

Tradeoffs

- Requires representative training and evaluation data
- Adds development, testing, and maintenance cost
- May need to be repeated when the base model changes

Recommended sequence

1. Start with prompting and evals
2. Add retrieval or tools for knowledge
3. **Fine-tune only for persistent behavior gaps**

Modern guidance on Fine-Tuning

- Early LLM applications often relied on fine-tuning
- No longer the first choice
- Modern AI platforms emphasize:
  - Prompt engineering
  - Evals
  - Retrieval Augmented Generation (RAG)
  - Tool use
- Current status
  - Viewed as a specialized optimization technique rather than a default solution.
- Modern Workflow
  - Prompting → Evals → RAG/Tools → Fine-Tuning (if needed)

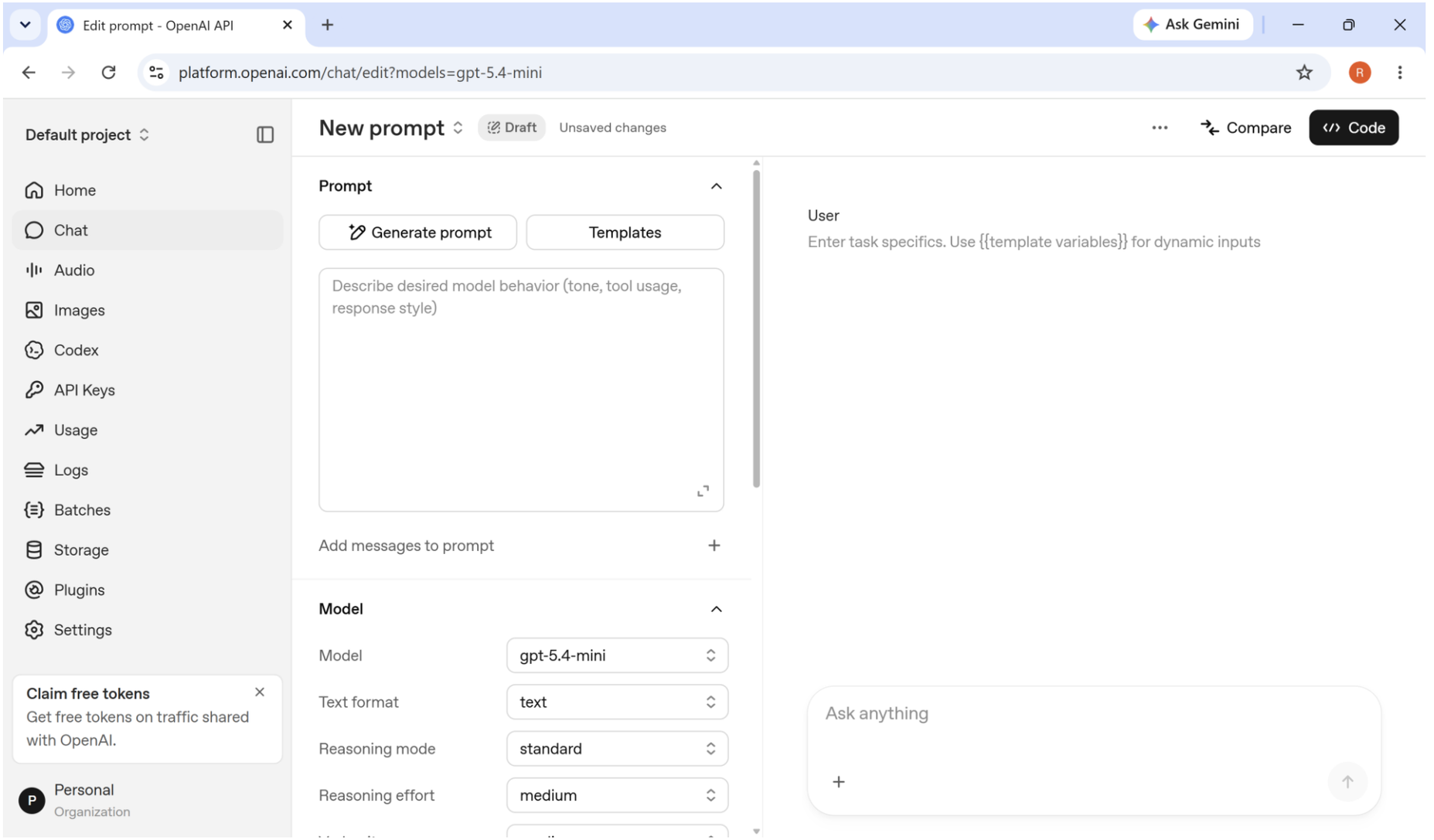
Prompt engineering

- GPT models are prompt-based
  - Input text is entered into prompt
  - Response from model represents completion of input
- Results are very dependent upon prompt
  - Obtaining high quality results can be difficult
  - Requires experience and intuition
- “More art than science”
  - https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/prompt-engineering

Prompt fundamentals

- Model uses the prompt to predict next sequence of words
  - Based on training data
  - Response consists of most likely prediction
- Prompt sections
  - System
    - Sets the model's overall instructions or behavior
  - User
    - Contains the user's questions, requests or input
  - Assistant
    - Contains AI generated response
    - Can also be provided as input for how you want the AI to respond
- Chat completion API
  - Prompt sections sent as array of dictionaries with associated roles

OpenAI Playground



System message

- Specified at the beginning of prompt
- Provides context/instructions for model
- Common to specify
  - Assistant personality
    - E.g. “You are a ***polite*** AI assistant…”
  - Format of responses
    - E.g., “You are an AI assistant that likes to send messages that rhyme…
  - Answers that should/shouldn’t be given
    - E.g., If you're unsure of an answer, you can say "I don't know"

Role prompting

- Technique to control response
  - Style
  - Tone
  - Depth
- Can enhance response
  - Quality and accuracy
- Examples

  “System: You are a computer scientist.

  User: Explain bubblesort”

Prompt components

- Instructions
  - Tell the model what to do
- Primary content
  - Text to be processed/transformed by model
- Cue
  - Help focus the model to provide desired output
- Supporting content
  - Secondary content that affects output of model
- At least one must be included
  - All others are optional
  - Multiple components usually supplied

Prompt components example

- Instructions
  - Write an email inviting everyone to attend the Generative AI course
- Primary content
  - Course description
- Cue
  - Bullet points with a one sentence summary
- Secondary content
  - August 25-26, Virtual delivery

Instruction prompting

- Place at the *beginning* of prompt
  - Before examples and context
  - Produces higher quality results
- Example
  - “Explain the main causes of inflation in 5 bullet points, use plain English, and give one real-world example for each cause.”
  - “Instruction: Answer every question in exactly one sentence and begin with “ANSWER:”

Few-shot learning

- Provide a few examples (shots) of what is desired from model
  - Usually in the form of input-output pairs
- Enables model to “learn” from provided examples
  - Only affects current chat
  - Does not permanently improve model
- Zero-shot learning
  - No examples provided

Zero-shot learning example

- System:
  - “You are a helpful assistant”
- User:
  - “tomato”
- Assistant:
  - “Tomato is a fruit that is commonly used as a vegetable in cooking. It is a good source of vitamins A and C, and is used in a variety of dishes including salads, sauces, soups, and stews. Tomatoes come in many different varieties, including cherry tomatoes, beefsteak tomatoes, and Roma tomatoes.”

Few-shot learning example

- System:
  - “You are a helpful assistant”
- User:

  “apple: fruit

  orange: fruit

  broccoli: vegetable

  cauliflower: vegetable

  tomato:”
- Assistant: (output from chat)
  - “fruit”

Few-shot learning example w/Assistant input

- User:

  “apple”
- Assistant:
  - “fruit”
- User:
  - “orange”
- Assistant:
  - “fruit”
- User:
  - “broccoli”
- Assistant:
  - “vegetable”
- User:
  - “tomato”

Chain of thought prompting

- Instruct model to
  - Take a step-by-step approach
  - Explain reasoning
- Benefits
  - Reduces inaccuracies
  - Enables assessing quality of response
- Example:
  - “Which presidential administration inherited the biggest surplus and left office with the biggest deficit? Take a step-by-step approach in your response, cite sources and give reasoning before sharing final answer in the below format: ANSWER is:”

Grouding context

- Provide the model with data
  - Referred to as “grounding data”
  - Highly relevant data minimizes possibility of errors
- Tell model to generate responses from that grounding data
- Example: Use model to answer customer service query
  - Decompose customer service data into chunks
  - Rank by relevance to different customer service issues
  - Find chunks corresponding to issue raised by new query
  - Provide model with relevant chunks to generate response
    - Optionally add historical data and agent input

S’s of prompt engineering

- Short
  - Prompt should be concise
  - Bad Example: "I am building a small hobby project and I really need you to please create a basic function for me that takes an array and finds the maximum number in it if that is okay."
  - Good Example: "Find the maximum integer in a given list."
- Surround
  - Provide the appropriate context
  - Bad Example: “Analyze this: *The company grew 20%”*
  - Good Example: “You are analyzing a company's annual report. The company grew revenue by 20% last year. Explain what this growth could indicate about its business performance, and identify two additional metrics we should examine"

S’s of prompt engineering

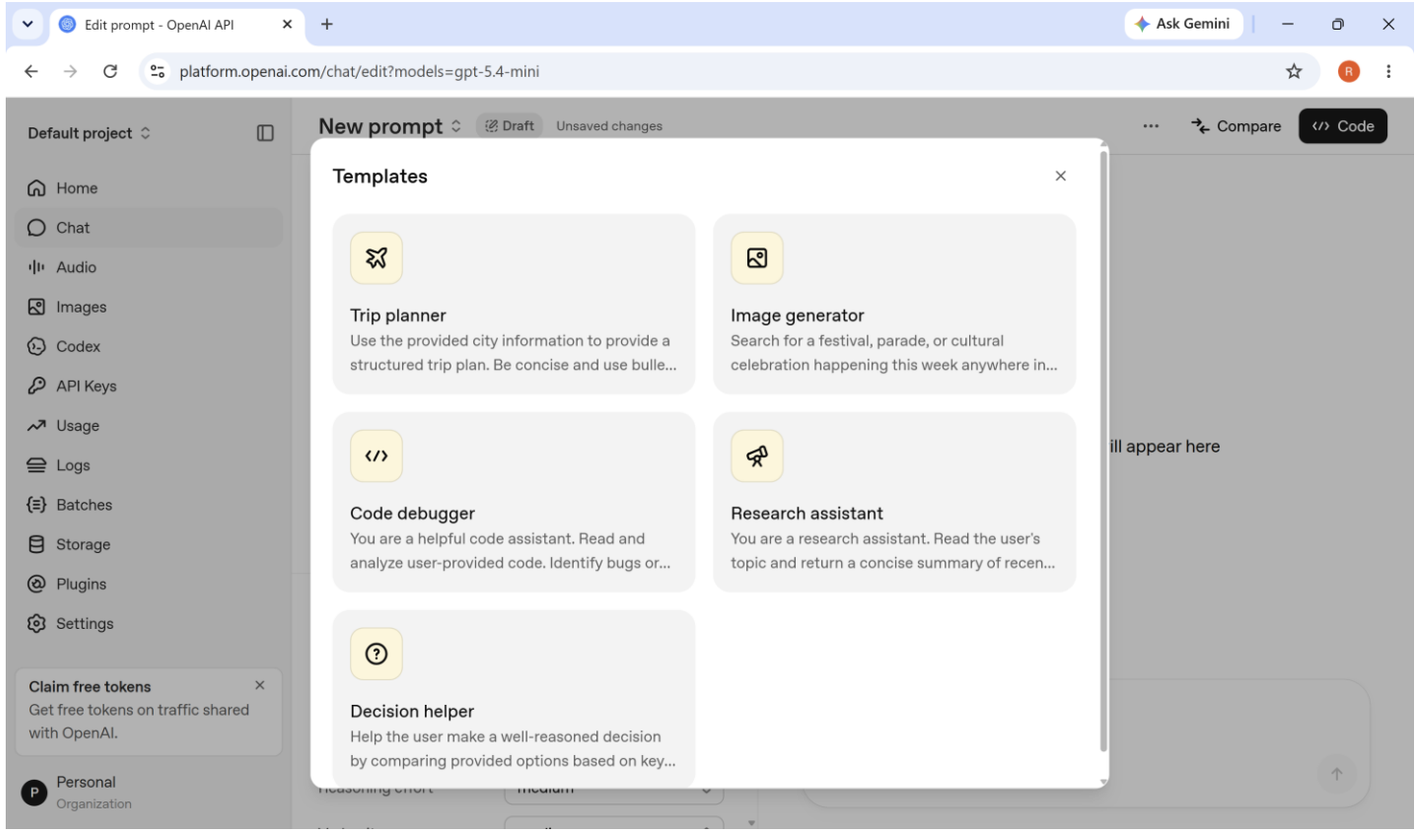
- Single
  - Prompt with single task or question
  - Bad Example: "Summarize this article, translate it, create a quiz, and write a social-media post."
  - Good Example: "Summarize this article in five bullet points.“
- Specific
  - Prompt should be clear and complete
  - Bad Example: "Fix the error in this code."
  - Good Example: "Fix the null dereference in parseUser() while preserving the current string return type."

Additional best practices

- Descriptiveness
  - Use analogies
- Repetition
  - Provide instructions before and after primary content
  - Use cues
- Ordering
  - Experiment with different ordering strategies
  - Be aware of recency bias
- Alternatives
  - Suggest alternative if task can’t be completed

Prompt templates

- Built-in templates available through the OpenAI Playground
  - Click *Templates* button from Chat page



Prompt templates

- Summarization
  - Summarize the following text in a concise paragraph:
    [Insert text]
- Content generation
  - Write a [type of content, e.g., blog post, email, product description] about [topic]. The target audience is [describe audience]. Include [specific elements or requirements].
- Q/A
  - Classify the following text into one of the following categories: [list categories].
    Text: [Insert text]

Prompt templates

- Rewrite/paraphrase
  - Rewrite the following sentence/text in simpler language:
    [Insert sentence/text]
- Extract information
  - Extract all [type of information, e.g., dates, names, locations] from the following text: [Insert text]
- Code generation
  - Write a [program/script/function] in [programming language] that does the following: [describe requirements].

Prompt templates

- Format conversion
  - Convert the following [type, e.g., CSV, text, JSON] to [desired format].
    Data: [Insert data]
- Analyze sentiment
  - What is the sentiment (positive, negative, neutral) of the following text?
    [Insert text]
- Role play/expert emulation
  - You are an expert [profession, e.g., doctor, lawyer, teacher]. Answer the following question as if you are advising a [client/patient/student]:
    [Insert question or scenario]

Temperature and Top_p

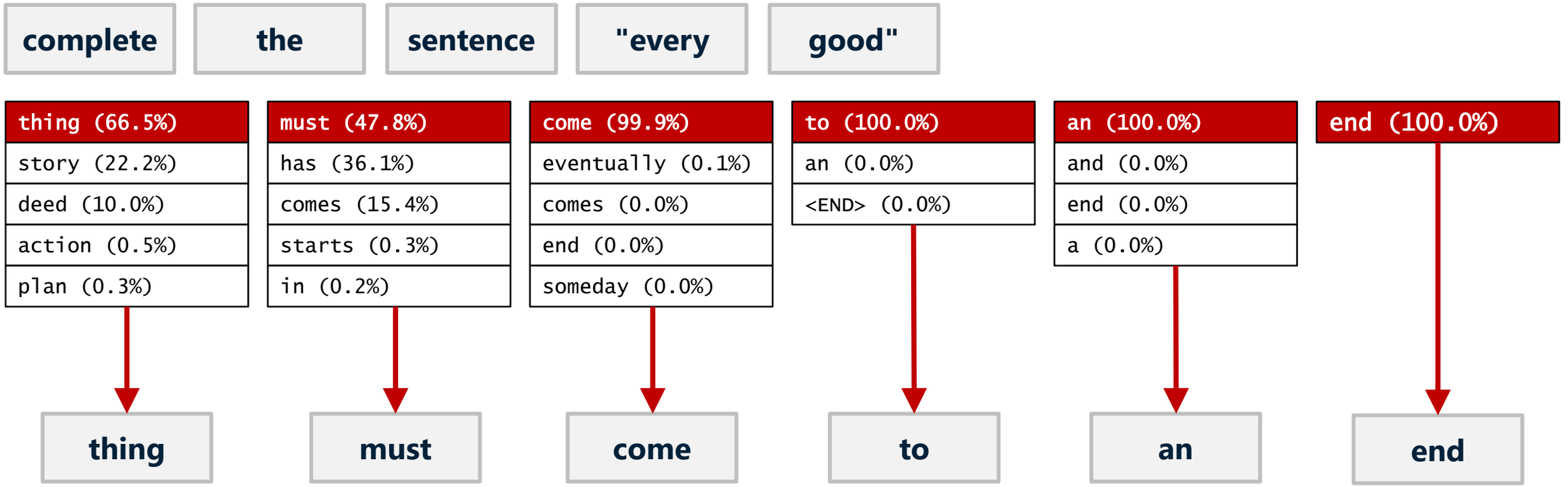
- Settings controlling randomness of output
- Higher values increase randomness (creativity) of output
  - Setting too high may generate results that do not make complete sense
- Lower values result in more predictable (deterministic) output
  - Setting both to zero may still give not give completely deterministic output
  - Randomness in GPU calculations may result in different results
- Best practice to modify only one

Temperature and Top_p

- Temperature
  - Range 0-2
    - 0 = mostly deterministic
    - 1 = default on many platforms
    - 2 = very random/creative
  - Specifies probably threshold for *individual* token
  - Model selects token that exceeds threshold
- Top_p
  - Range 0-1
  - Specifies probability threshold for *combination* of tokens
  - Model selects from tokens that cumulatively meet or exceed threshold
    - Restricts possible tokens based on cumulative rather than individual probabilities

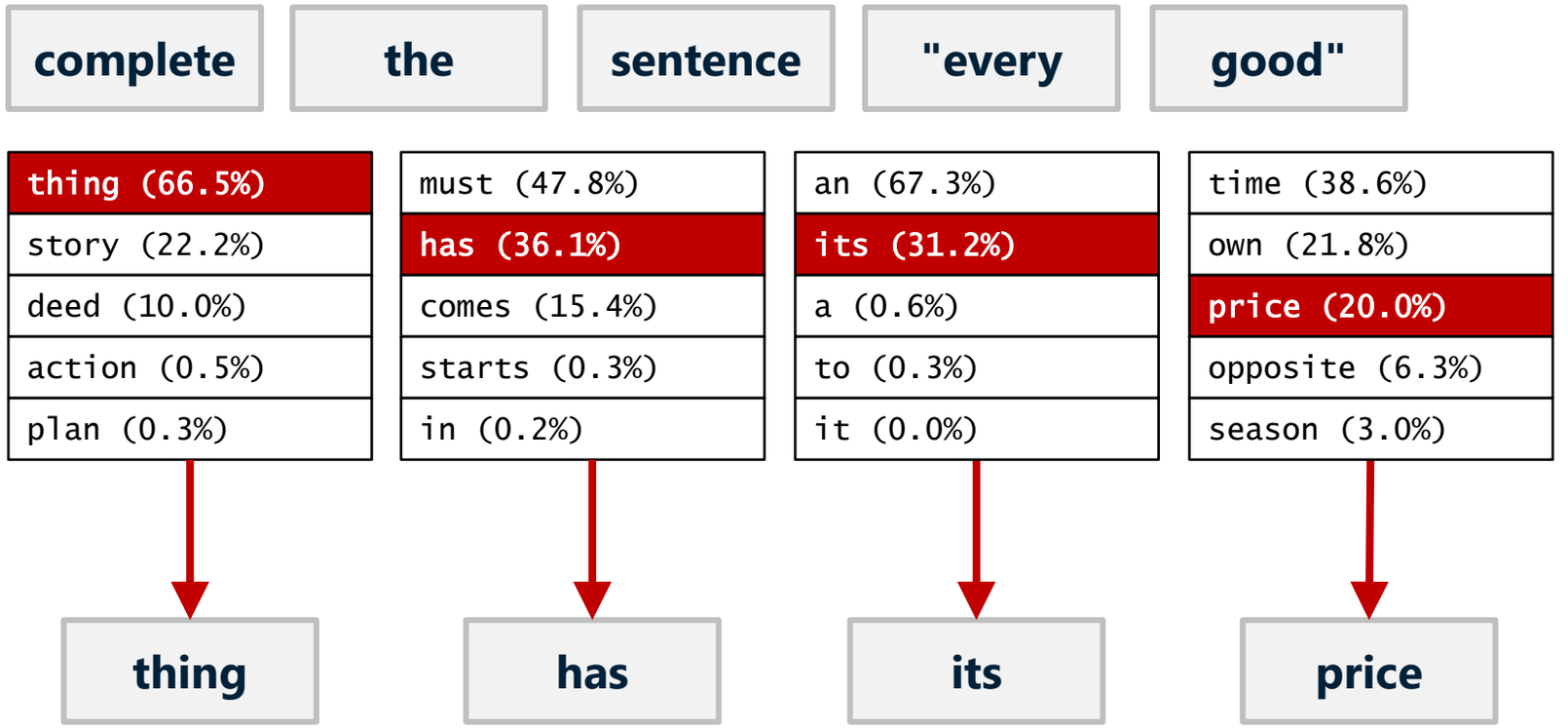
How LLMs Work

`temperature=0.0 (greedy sampling)`



How LLMs Work, Cont.

`temperature=1.0 (pure sampling)`



Top_p completion example

- Value=0.9
- Complete the sentence “John arrived home by”
- Top choices and probabilities
  - Car=60%
  - Train=20%
  - Walking=10%
  - Bike=5%
  - Bus=5%
- Combined probabilities of “Car”, “Train” and “Walking” = Top_P threshold
  - Model will randomly select from these three choices

Nucleus Sampling

- Controlled by **top_p** (default=1.0)
- Lower values restrict tokens in the sampling pool by including minimum number of tokens whose cumulative probability >= **top_p**